

A

BUILDING A WINDOWS DOMAIN NETWORK FOR TESTING



Several chapters in this book make reference to a Windows domain network you can use for testing purposes. While you don't need to set up such a network to follow along with the chapters, you can use it to run the examples, then alter the provided commands to observe different outcomes. If you don't already have a suitable Windows domain network on hand to use for testing, this appendix will walk you through setting one up with virtual machines.

Running Windows in a virtual machine has many advantages. First, it gives you complete flexibility to configure (or misconfigure) Windows without compromising the security of your everyday installation. Virtualization platforms typically allow you to snapshot your virtual machines so you can roll them back to a known good state if something goes wrong. You can also isolate network traffic to prevent it from affecting

other systems on the same network. Lastly, you can use a virtual machine to run Windows in a non-Windows environment.

The domain configuration steps use PowerShell whenever possible. Note that, unless otherwise stated, you must run all of these PowerShell commands as an administrator user.

The Domain Network

Figure A-1 is a diagram of the network we'll build. For more information about the structure of domain networks in general, consult **Chapter 10**.

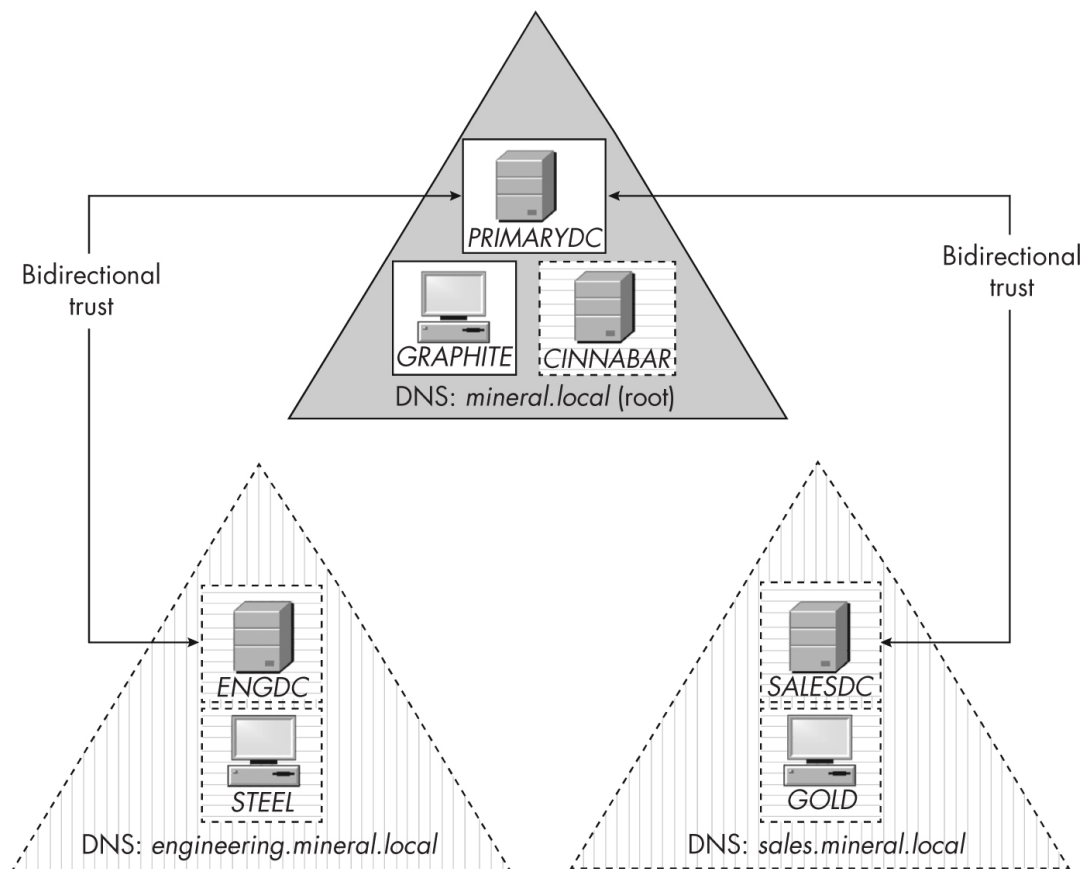


Figure A-1: The domain network configuration

The network includes a forest made up of three domains. The root DNS name for the forest is *mineral.local*, and its two child domains are *engineering.mineral.local* and *sales.mineral.local*. To create a minimal functional domain for testing, you need only *PRIMARYDC*, which is the root domain controller, and *GRAPHITE*, a workstation joined to the domain. Anything included in dotted lines is optional. The next sections will

show you how to set up the domain network and configure virtual machines for each of the Windows systems you want to include.

Installing and Configuring Windows Hyper-V

We'll set up the Windows domain network using Hyper-V, which is virtualization software that comes for free on 64-bit versions of Windows Professional, Enterprise, and Education. If you're not running Windows or don't want to use Hyper-V, another good free option is Oracle's VirtualBox (<https://www.virtualbox.org>).

To install Hyper-V and its tools, start an administrator PowerShell console and run the following command. Make sure to restart the system after installation:

```
PS> Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Hyper-V -All
```

The next step is to configure a new network for the virtual machines, as shown in [Listing A-1](#). This allows you to have complete control over all aspects of the network configuration for the domain network and isolates it from your real network.

```
PS> New-VMSwitch -Name "Domain Network" -SwitchType Internal
PS> $index = (Get-NetAdapter |
Where-Object Name -Match "Domain Network").ifIndex
PS> New-NetIPAddress -IPAddress 192.168.99.1 -PrefixLength 24
-InterfaceIndex $index
PS> New-NetNat -Name DomNAT -InternalIPInterfaceAddressPrefix 192.168.99.0/24
```

Listing A-1: Creating a new virtual machine network switch

We first create a new switch for the domain network using the `New-VMSwitch` command, which you need to do only once during this initial configuration process. We give the switch the name "Domain Network" and set its type to `Internal`, which means it's a virtual network that can communicate with the virtual machine host.

Next, we need to assign the virtual network adapter that was created for the switch with an IP address. The `Get-NetAdapter` command lists all network adapters and finds the unique index number for the adapter for our domain network. We then assign the IP address of 192.168.99.1 to the adapter, with a subnet prefix of 24 bits (perhaps more commonly seen as the subnet mask 255.255.255.0). You're welcome to set the IP address to any value you like, but keep in mind that if you change the address, you'll also need to update it throughout the rest of this appendix.

The final step is to set up *network address translation (NAT)* for the IP address using the `New-NetNat` command. This will allow computers on the network to access the internet by setting their default gateway to the adapter's IP address, in this case 192.168.99.1.

NOTE

This configuration doesn't set up a Dynamic Host Configuration Protocol (DHCP) server to automatically assign IP addresses to computers on the network. As the network is so small, we'll just statically assign IP addresses to the computers as we go.

Creating the Virtual Machines

Table A-1 lists the virtual machines we'll set up, along with the operating system type and IP address for each. I'll walk through setting up the *PRIMARYDC*, *GRAPHITE*, and optional *SALESDC* virtual machines. The other virtual machines in the table are completely optional; if you want to create them, you can replace the specified values in the sections on setting up each virtual machine with the appropriate values from the table.

Table A-1: Virtual Machine Names and IP Addresses

Virtual machine name	Operating system	IP address
<i>PRIMARYDC</i>	Windows Server	192.168.99.10

Virtual machine name	Operating system	IP address
GRAPHITE	Windows Professional or Enterprise	192.168.99.50
CINNABAR	Windows Server	192.168.99.20
SALESDC	Windows Server	192.168.99.110
GOLD	Windows Professional or Enterprise	192.168.99.150
ENGDC	Windows Server	192.168.99.210
STEEL	Windows Professional or Enterprise	192.168.99.220

Microsoft provides trial editions of Windows Enterprise and Windows Server as virtual machines. I'd recommend using your favorite search engine to find the latest links on Microsoft's website. For each machine, install the correct Windows version and then use PowerShell to configure it.

To use Windows virtual machines with Hyper-V, you'll need the installation media and license keys for Windows Professional or Enterprise and Windows Server. A common way to get access to these is through a Microsoft Visual Studio subscription. The versions of Windows and Server you use won't matter for the topics we'll discuss.

NOTE

Server installations include a long-term service branch that comes with the Windows desktop and a more up-to-date version, called a server core version, that has only a command line. As we'll configure the server installation with PowerShell, either version will work.

However, if you're more comfortable with a GUI, use the long-term service branch with a desktop instead.

Listing A-2 defines the function we'll use to do most of the work of setting up a virtual machine, `New-TestVM`.

```
PS> function New-TestVM {  
    param(  
        [Parameter(Mandatory)]  
        [string]$VmName,  
        [Parameter(Mandatory)]  
        [string]$InstallerImage,  
        [Parameter(Mandatory)]  
        [string]$VmDirectory  
    )  
    ❶ New-VM -Name $VmName -MemoryStartupBytes 2GB -Generation 2  
    -NewVHDPATH "$VmDirectory\$VmName\$VmName.vhdx" -NewVHDSIZEBytes 80GB  
    -Path "$VmDirectory" -SwitchName "Domain Network"  
    ❷ Set-VM -Name $VmName -ProcessorCount 2 -DynamicMemory  
    ❸ Add-VMScsiController -VMName $VmName  
    Add-VMdvdDrive -VMName $VmName -ControllerNumber 1 -ControllerLocation 0  
    -Path $InstallerImage  
    $dvd = Get-VMdvdDrive -VMName $VmName  
    Set-VMFirmware -VMName $VmName -FirstBootDevice $dvd  
}
```

Listing A-2: Defining the New-TestVM function

The `New-TestVM` function takes the name of the virtual machine so it can create the path to the DVD image to install and the base directory for the virtual machine's assets. We start by calling the `New-VM` command to create the virtual machine ❶. We set its memory to 4GB and create an 80GB virtual hard disk. (You can increase these sizes if you like.) We also assign the default network adapter to use the "Domain Network" switch we created in **Listing A-1**.

Next, we use the `Set-VM` command to configure some virtual machine options not exposed through `New-VM` ❷. We assign two CPUs to the virtual machine, as I find modern versions of Windows struggle with only one CPU.

You can increase the number of CPUs if your base machine has many CPU cores.

We also enable dynamic memory. This allows Windows to scale the virtual machines' memory usage as needed. I've found that typically a server installation uses only around 2GB of memory when running, but it could be more, especially for clients. Dynamic memory can both increase and decrease allocated memory as needed.

Finally, we set up a DVD drive on a virtual SCSI controller and assign the DVD image to it ③. We'll use this as the primary boot drive, so we can install the operating system from the DVD image.

We now need to create each virtual machine using the function we defined and start the installation process.

The PRIMARYDC Server

The *PRIMARYDC* machine is a Windows server that will act as the root domain controller for our forest. In [Listing A-3](#), we start by creating the virtual machine as an administrator.

```
PS> New-TestVM -VmName "PRIMARYDC" -InstallerImage "C:\iso\server.iso"  
-VmDirectory "C:\vms"  
PS> vmconnect localhost PRIMARYDC  
PS> Start-VM -VmName "PRIMARYDC"
```

Listing A-3: Creating and starting the PRIMARYDC virtual machine

We install the *PRIMARYDC* virtual machine from the DVD image file *C:\iso\server.iso* and create the virtual machine in the *C:\vms* directory. This should create a new directory under the virtual machine directory for the *PRIMARYDC* server's files, which allows us to separate our resources for each of our virtual machines. Next, we start the virtual machine's user interface so we can interact with the installation process, and then start the virtual machine.

Now that you can interact with the virtual machine, you can follow the installation steps as for any other Window Server installation. I won't provide detailed instructions for this, as it's mostly a case of selecting your region and the installation drive and following the default process.

When asked for the *Administrator* user's password during the installation, you can set anything you like, but in this book I've assumed it will be set to `Passw0rd`. As this is a weak password, do not expose these virtual machines to a network where untrusted users can access them. However, for testing and demonstration purposes, having easily memorable passwords is usually a good idea.

Once you've gained access to either a desktop (if using the long-term service branch version of the server) or a command line (if using the server core version), you can finish the basic setup. All subsequent PowerShell commands will be run on the VM itself, not the host. First start an administrator copy of PowerShell to run the commands in [Listing A-4](#).

```
PS> $index = (Get-NetAdapter).ifIndex
PS> New-NetIPAddress -InterfaceIndex $index -IPAddress 192.168.99.10
    -PrefixLength 24 -DefaultGateway 192.168.99.1
PS> Set-DnsClientServerAddress -InterfaceIndex $index -ServerAddresses 8.8.8.8
```

Listing A-4: Setting up the PRIMARYDC virtual machine network

As the network switch we created earlier doesn't include support for DHCP, it won't automatically assign an IP address during installation. Thus, we need to set up the network with static IP addresses. [Listing A-4](#) starts by setting the IP address of the network adapter; you should use the IP address from [Table A-1](#) for the virtual machine you're configuring. The `DefaultGateway` parameter should be the IP address you set on the host in [Listing A-1](#) so that traffic can be routed to the external network.

You'll also need to specify a DNS server address for the network adapter. In [Listing A-4](#) we set this to the address of the public Google DNS server, 8.8.8.8. If you know the IP address of your internet provider or another preferred DNS server, use that instead. Once we've finished setting up the

domain controller, we'll no longer need this DNS server, as the domain controller has its own DNS server.

You should now be able to access an external network. At this point, you might need to activate your copy of Windows Server if you're not using a trial version. You'll also want to ensure that the copy of Windows is up to date, including all security patches. While the network will isolate the virtual machines from external networks to a degree, this doesn't mean they can't be compromised, so it's best to be certain.

Next, rename the computer using the `Rename-Computer` command, as shown in [Listing A-5](#).

```
PS> Rename-Computer -NewName "PRIMARYDC" -Restart
```

Listing A-5: Renaming the computer

This name will be used on the domain network, so it helps to have memorable names. Replace *PRIMARYDC* with your own name if you prefer.

Once you've renamed the computer, you need to configure the server as the domain controller for the *mineral.local* domain. Log in to the server as an administrator and run the commands in [Listing A-6](#).

```
PS> Install-WindowsFeature AD-Domain-Services
PS> Install-ADDSForest -DomainName mineral.local -DomainNetbiosName MINERAL
    -InstallDns -Force
SafeModeAdministratorPassword: *****
Confirm SafeModeAdministratorPassword: *****
```

Listing A-6: Installing and configuring the Active Directory domain services

First, we install the `AD-Domain-Services` feature. This feature installs the Active Directory server and associated services to run the server as a domain controller. Next, we run the `Install-ADDSForest` command to set up the forest and create the root domain. We specify the DNS name of the domain, which in this case is *mineral.local*. We also specify the simple name

of the domain as *MINERAL* and request that a local DNS server be installed. Active Directory can't work without a DNS server, and as this is an isolated network, it makes sense to run the DNS server on the domain controller server.

When setting up the forest, you'll be asked to specify a safe-mode administrator password. This password allows you to recover the Active Directory database. In such a small, non-production domain, you're unlikely to need this feature, but you should still specify a password you can remember. You're likely to see a few warnings during the installation; you can safely ignore these. Once the command has completed, the server will reboot automatically.

When it has finished rebooting you should reauthenticate to the server, but make sure to use the username *MINERAL\Administrator* so that you can use the domain administrator account. The password for the domain administrator should be the same as the one you initially configured when installing the server. Then, start an instance of PowerShell and run the commands in [Listing A-7](#) to do some basic user setup.

```
❶ PS> Set-ADDefaultDomainPasswordPolicy -Identity mineral.local
    -MaxPasswordAge 0
❷ PS> $pwd = ConvertTo-SecureString -String "Passw0rd1" -AsPlainText -Force
PS> New-ADUser -Name alice -Country USA -AccountPassword $pwd
    -GivenName "Alice Bombas" -Enabled $true
PS> $pwd = ConvertTo-SecureString -String "Passw0rd2" -AsPlainText -Force
PS> New-ADUser -Name bob -Country JP -AccountPassword $pwd
    -GivenName "Bob Cordite" -Enabled $true
❸ PS> New-ADGroup -Name 'Local Resource' -GroupScope DomainLocal
PS> Add-ADGroupMember -Identity 'Local Resource' -Members 'alice'
PS> New-ADGroup -Name 'Universal Group' -GroupScope Universal
PS> Add-ADGroupMember -Identity 'Universal Group' -Members 'bob'
PS> New-ADGroup -Name 'Global Group' -GroupScope Global
PS> Add-ADGroupMember -Identity 'Global Group' -Members 'alice','bob'
```

Listing A-7: Configuring the domain password policy and adding users and groups

First, we set the domain's password policy to prevent passwords from expiring ❶. There's nothing worse than coming back to your virtual machines after a few months and being faced with changing the passwords, which you immediately forget.

NOTE

Even though the default password expiry for a new domain is 42 days, Microsoft no longer recommends having forced password expiry enabled. This is because making users change their password frequently can cause more harm than good by encouraging them to use trivial passwords, so they don't forget them.

We then create two domain users, *alice* and *bob*, assigning each of them a password ❷. We also set a few Active Directory attributes for each user: specifically, their name and country. I've summarized the values to specify in **Table A-2**. Of course, you can set the names and values to anything you prefer.

Table A-2: Default Users for the Root Domain

Username	Given name	Country	Password
<i>alice</i>	Alice Bombas	USA	Passw0rd1
<i>bob</i>	Bob Cordite	JP	Passw0rd2

The final task in **Listing A-7** is to create three Active Directory groups ❸, one for each group scope. We also assign the two users to a combination of these groups.

The GRAPHITE Workstation

With the domain controller configured, we can now set up a workstation. Run the script in **Listing A-8** to create the virtual machine, as we did with *PRIMARYDC*.

```
PS> New-TestVM -VmName "GRAPHITE" -InstallerImage "C:\iso\client.iso"
-VmDirectory "C:\vms"
PS> vmconnect localhost GRAPHITE
PS> Start-VM -VmName "GRAPHITE"
```

Listing A-8: Creating and starting the GRAPHITE virtual machine

In this case, you'll use a disk image of Windows Professional or Enterprise, rather than a server installation. Any currently supported version of Windows 10 or greater is sufficient. Proceed with the installation as you normally would, creating the machine's username and password. This book assumes you'll use the username *admin* and a password of *Passw0rd*, but you can pick any username and password you prefer.

Listing A-9 sets up the network, as in **Listing A-4**.

```
PS> $index = (Get-NetAdapter).ifIndex
PS> New-NetIPAddress -InterfaceIndex $index -IPAddress 192.168.99.50
-PrefixLength 24 -DefaultGateway 192.168.99.1
PS> Set-DnsClientServerAddress -InterfaceIndex $index
-ServerAddresses 192.168.99.10
PS> Resolve-DnsName primarydc.mineral.local
```

Name	Type	TTL	Section	IPAddress
----	----	---	-----	-----
primarydc.mineral.local	A	3600	Answer	192.168.99.10

```
PS> Rename-Computer -NewName "GRAPHITE" -Restart
```

Listing A-9: Setting the domain DNS server and checking that it resolves

The only difference here is that we configure the DNS server to use the one we installed on the domain controller at 192.168.99.10. You can verify that the DNS server is working correctly by attempting to resolve the *primarydc.mineral.local* server address. You should also be able to resolve internet domain names, as the domain controller will forward the requests onward.

Again, once you've configured this network, you'll want to ensure that you've activated your Windows installation if necessary and downloaded any updates. If desired, you can rename the workstation to your chosen name before continuing.

In [Listing A-10](#), we join the workstation to the domain.

```
PS> $creds = Get-Credential
PS> Add-Computer -DomainName MINERAL -Credential $creds
WARNING: The changes will take effect after you restart the computer GRAPHITE

PS> Add-LocalGroupMember -Group 'Administrators' -Member 'MINERAL\alice'
PS> Restart-Computer
```

Listing A-10: Joining the GRAPHITE workstation to the domain

The first thing we need are the credentials for a user in the domain. As I explained in [Chapter 11](#), this user doesn't need to be a domain administrator; it can be a normal user. For example, you can enter the credentials for the *alice* user when prompted by the `Get-Credential` command's GUI.

Next, we call the `Add-Computer` command to join the workstation to the *MINERAL* domain with the user's credentials. If this succeeds, it will print a warning telling you to restart the computer. However, don't restart it just yet; you first need to add a domain user, such as *alice*, to the local *Administrators* group using the `Add-LocalGroupMember` command. If you don't do this step, you'll subsequently have to authenticate to the workstation using either a domain administrator or the original local administrator account. Adding a user to this group allows you to authenticate as that user and be a local administrator. Once this is done, you can reboot.

That's all there is to setting up a workstation. You can configure the rest of the workstation's settings through the group policy on the domain controller. Once the workstation has restarted, you should be able to authenticate as any domain user.

The SALESDC Server

The *SALESDC* virtual machine is a Windows server that serves a domain controller for the *sales.mineral.local* domain within the forest. Setting up this machine (or its sibling, *ENGDC*) is optional: you don't need multiple domain forests to run most of the examples in this book. However, it will allow you to test different behaviors.

Listing A-11 includes the same commands as those run for the *PRIMARYDC* virtual machine, with different values.

```
PS> New-TestVM -VmName "SALESDC" -InstallerImage "C:\iso\server.iso"
-VmDirectory "C:\vms"
PS> vmconnect localhost SALESDC
PS> Start-VM -VmName "SALESDC"
```

Listing A-11: Creating and starting the SALESDC virtual machine

Follow the normal installation process, and when asked for the *Administrator* user's password, set it to anything you like. In this book, I've assumed it will be set to `Password`.

Listing A-12 configures the virtual machine's network using the DNS server on *PRIMARYDC*.

```
PS> $index = (Get-NetAdapter).ifIndex
PS> New-NetIPAddress -InterfaceIndex $index -IPAddress 192.168.99.110
-PrefixLength 24 -DefaultGateway 192.168.99.1
PS> Set-DnsClientServerAddress -InterfaceIndex $index
-ServerAddresses 192.168.99.10
PS> Rename-Computer -NewName "SALESDC" -Restart
```

Listing A-12: Setting up the SALESDC virtual machine network

It's crucial that the DNS client point to the root domain controller when creating a new domain in the forest so that you can resolve the root domain information. Once you've renamed the computer, you'll need to

configure the server as the domain controller for the *sales.mineral.local* domain. Log in to the server as an administrator and run the commands in [Listing A-13](#).

```
PS> Install-WindowsFeature AD-Domain-Services
PS> Install-ADDSDomain -NewDomainName sales -ParentDomainName mineral.local
-NewDomainNetbiosName SALES -InstallDns -Credential (Get-Credential) -Force
SafeModeAdministratorPassword: *****
Confirm SafeModeAdministratorPassword: *****
```

Listing A-13: Installing and configuring the Active Directory domain services for a child domain

Here, you first install the `AD-Domain-Services` feature as before, then run the `Install-ADDSDomain` command to create a new domain in an existing forest. You'll be prompted for the safe-mode password, as with the root domain. You must also specify an administrator account in the root domain to establish the trust relationship. You can use the existing *MINERAL\Administrator* account for this.

If this succeeds, the server should reboot. When you can reauthenticate as the *SALES\Administrator* user, you can verify that you've set up a trusted connection by using the `Get-ADTrust` command, as shown in [Listing A-14](#).

```
PS> Get-ADTrust -Filter * | Select Target, Direction
Target          Direction
-----
mineral.local    BiDirectional
```

Listing A-14: Verifying the trust relationship between the SALES and root domains

You should see a single entry for the root *mineral.local* domain. If the command fails, wait a few minutes for everything to start and retry.

At this point, you can add your own users and groups to the *SALES* domain, which will be separate from the root domain, although the users should be able to authenticate across domains due to the configured trust relationship. You can also install your own workstations using the steps outlined for *GRAPHITE*, making sure to specify the DNS server using the *SALES*SDS IP address.

You can also create a separate engineering domain in the forest, or anything else you'd like. Just repeat these steps, changing the IP addresses and names you assign. You should then have a basic domain and forest configuration with which to run the examples in this book.

While we've configured every system you'll need for the book, you are free to configure and customize these domains further if you wish. Bear in mind that changing certain configurations, such as names or passwords, might change the input you'll need to provide in the book's examples.